

ZIPPY 서비스 개발 구현 결과

가족이 등록한 돌봄 정보를 일정과 담당 배정으로 연결하고, 처리 결과를 앱 알림과 Firebase Cloud Messaging으로 전달하도록 구현하였다. 이 문서는 주요 화면과 실제 소스 코드를 기준으로 서비스 전체 구현 범위를 정리한다.

1 서비스 구현 범위

정보 입력 사진 및 텍스트 →	AI 분석 OCR 및 항목 분류 →	일정 반영 아이 일정 저장 →	담당 배정 후보 추천 및 확정 →	알림 전달 DB 알림 및 FCM →	상태 확인 홈 일정 배정 화면
---------------------	------------------------	---------------------	-----------------------	------------------------	---------------------

주요 기능은 별도 화면으로 분리되어 있지만, 등록된 돌봄 정보와 일정, 배정, 알림 데이터가 하나의 처리 흐름으로 연결된다.

구현 영역	구현 내용	사용자 결과
알림장 분석	사진 또는 텍스트를 입력받아 OCR과 AI 구조화를 실행하고 일정 준비물 숙제 항목을 생성	검토 항목 생성
일정 관리	아이 일정과 개인 일정을 저장하고 Google Calendar와 Outlook 연동 상태를 관리	통합 일정 확인
돌봄 배정	가족 일정 충돌과 기존 돌봄 건수를 비교해 담당 후보를 추천하고 요청 수락 확정 처리	담당 상태 추적
알림 처리	기능 이벤트를 DB 알림으로 기록하고 등록된 기기 토큰으로 FCM 푸시 전송	앱 외부 알림 수신
홈 상태	오늘 일정과 배정 및 진행 상태를 한 화면에 요약하고 관련 기능으로 이동	현재 상황 확인

구현 기술

영역	기술 및 역할
프론트엔드	React TypeScript Capacitor 기반 화면과 모바일 알림 처리
백엔드	FastAPI 기반 API AI 처리 배정 로직 알림 이벤트 관리
데이터 및 연동	SQLite 데이터 저장 OpenAI 기반 분석 Firebase Cloud Messaging 푸시 전달

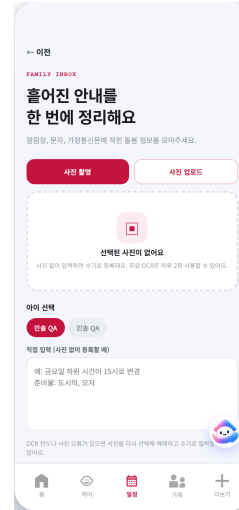
2 주요 화면 구현 결과

서비스의 핵심 흐름을 확인할 수 있는 홈 알림장 등록 일정 돌봄 배정 화면을 실제 실행 결과로 정리하였다.



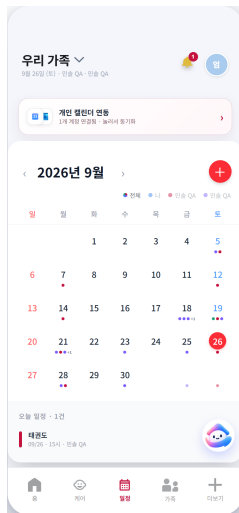
홈

오늘 일정과 담당 및 돌봄 진행 상태 요약



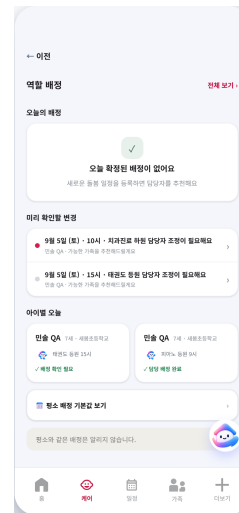
알림장 등록

사진 또는 텍스트 입력과 AI 분석 시작



일정

아이 일정과 가족 일정을 날짜별로 확인



돌봄 배정

추천 후보와 요청 수락 및 확정 상태 확인

3 프론트엔드 구조 및 구현

프론트엔드는 입력 결과를 일정과 배정 화면으로 연결하고, 네이티브 알림을 누르면 이벤트 종류에 맞는 화면으로 이동하도록 구성하였다.

구성	실제 소스	역할
화면과 상태	frontend src App.tsx	화면 전환 사용자 입력 부트스트랩 상태 관리
API 통신	frontend src api.ts	공통 요청 인증 토큰 응답 타입 처리
모바일 알림	frontend src nativeNotifications.ts	Capacitor 알림 권한 FCM 토큰 알림 액션 처리
네이티브 상태	frontend android LiveCareStatusPlugin.java	돌봄 진행 상태를 Android 지속 알림으로 표시

ACTUAL SOURCE CODE

입력 처리와 결과 화면 전환 frontend/src/App.tsx lines 1476-1493

```

1476 const result = captureFile ? await () => {
1477   const form = new FormData()
1478   form.append('file', captureFile)
1479   form.append('child_id', captureChild)
1480   form.append('source', captureSource)
1481   return upload<({ items: CareItem[], transcript: string, ocr_used_today: number, registered_child_schedules?:
      unknown[] })>(/intakes/photo', form)
1482 }(): await send<({ items: CareItem[], transcript?: string })>(/intakes', 'POST', {
1483   child_id: captureChild, raw_content: captureText.trim(), input_type: 'TEXT',
1484 })
1485 const registered = result.items.find(item => item.status === 'CONFIRMED' && item.child_schedule_id)
1486 if (registered) {
1487   const ranked = await api<({ suggestions: Suggestion[] })>(/items/ + registered.id + /suggestions')
1488   setItemId(registered.id); setSuggestions(ranked.suggestions)
1489 } else if (result.items[0]) selectReview(result.items[0])
1490 else setItemId(null)
1491 setCaptureFromPhoto(!captureFile)
1492 setCaptureTranscript(result.transcript ?? captureText.trim())
1493 setCaptureText(""); setCaptureFile(null); setCapturePreview(""); go(registered ? 'suggestion' : result.items[0]
  || captureFile ? 'review' : 'family')

```

알림장 입력을 API로 전송하고 분석 결과에 따라 검토 또는 배정 추천 화면으로 이동한다.

ACTUAL SOURCE CODE

알림 수신 후 관련 화면 이동 frontend/src/App.tsx lines 1134-1165

```

1134 useEffect() => {
1135   void setupNativeNotifications(action => {
1136     window.dispatchEvent(new CustomEvent('family-care-notification-action', { detail: action }))
1137   })
1138 }, []
1139 useEffect() => {
1140   if (me ? authenticated : void syncPushTokens)
1141   }, [me ? authenticated]
1142   useEffect() => {
1143     const onNativeAction = (event: Event) => {
1144       const { actionType, actionId } = (event as CustomEvent<{ actionType?: string, actionId?: string }>).detail ?? {}
1145       window.focus()
1146       if (actionType === 'ASSIGNMENT_REQUEST' && actionId) {
1147         setAssignmentId(actionId); setViewer(me ? member.id ?? viewer)
1148         history.pushState({ ...history.state, lgdScreen: 'tasks' }, ''); setScreen('tasks')
1149       } else if (actionType === 'ASSIGNMENT_RESULT') {
1150         setAssignmentId(actionId ?? null)
1151         history.pushState({ ...history.state, lgdScreen: 'assignments' }, ''); setScreen('assignments')
1152       } else if (actionType === 'CARE_SUGGESTION' && actionId) {
1153         void api<({ item: CareItem, suggestions: Suggestion[] })>(/items/ + actionId + /suggestions') then(result => {
1154           setItemId(result.item.id); setSuggestions(result.suggestions)
1155           history.pushState({ ...history.state, lgdScreen: 'suggestion' }, ''); setScreen('suggestion')
1156         }).catch(reportError)
1157       } else if (actionType === 'HANDOFF') {
1158         setViewer(me ? member.id ?? viewer)
1159         history.pushState({ ...history.state, lgdScreen: 'tasks' }, ''); setScreen('tasks')
1160       } else {
1161         history.pushState({ ...history.state, lgdScreen: 'notifications' }, ''); setScreen('notifications')
1162       }
1163     }
1164     window.addEventListener('family-care-notification-action', onNativeAction)
1165     return () => window.removeEventListener('family-care-notification-action', onNativeAction)

```

알림의 actionType을 기준으로 할 일 배정 추천 알림함 화면을 직접 연다.

4 백엔드 구조 및 구현

백엔드는 AI 입력 처리와 일정 구조화 담당 후보 추천 배정 확정 알림 생성을 기능 단위로 처리한다.

구성	실제 소스	역할
AI 처리	backend app ai.py	이미지 텍스트 추출과 일정 항목 구조화
확장 API	backend app extended.py	사진 음성 캘린더 결제 등 확장 기능 라우터
핵심 API	backend app main.py	일정 배정 인수인계 알림과 가족 데이터 처리
푸시 전달	backend app push.py	Firebase Admin 초기화와 FCM 메시지 전송
데이터 저장	backend app db.py	가족 일정 배정 알림 기기 토큰 저장

ACTUAL SOURCE CODE

사진 전송과 OCR 일정 구조화 backend/app/extended.py lines 358-375

```

358 @router.post("/intakes/photo", status_code=201)
359 def photo_intake(file: UploadFile, child_id: str = Form(...), source: str = Form(default="ALBUM")):
360     if source not in ["ALBUM", "CAMERA"]:
361         raise HTTPException(422, "source는 ALBUM 또는 CAMERA여야 합니다")
362     image = _read_file(file, 10 * 1024 * 1024)
363     mime = _image_mime(image)
364     with database() as db:
365         db.execute("BEGIN IMMEDIATE")
366         if not db.execute("SELECT 1 FROM child WHERE id = ? AND family_id = ?", (child_id, family_id)).fetchone():
367             raise HTTPException(404, "아이를 찾을 수 없습니다")
368         if _plan(db) == "FREE" and _used(db, "OCR") >= 2:
369             raise HTTPException(403, detail={"code": "OCR_DAILY_LIMIT", "message": "무료 OCR은 하루 2회입니다. 텍스트 입력 또는 Pro를 이용해주세요"})
370         _add_usage(db, "OCR", 1)
371     try:
372         extracted = ai.extract_image_text(image, mime)
373         parsed_items = ai.extract_schedule_items(extracted)
374         from main import IntakeCreate, store_intake
375         result = store_intake(IntakeCreate(raw_content=extracted, child_id=child_id, input_type="PHOTO_TRANSCRIPT"),
376                               parsed_items)

```

사진 입력을 검증한 뒤 OCR과 일정 항목 추출을 실행하고 저장 함수로 전달한다.

ACTUAL SOURCE CODE

담당자 최종 확정과 결과 알림 backend/app/main.py lines 1373-1394

```

1373 @app.post("/api/assignments/assignment_id/confirm")
1374 def confirm_assignment(assignment_id: str):
1375     """Let the primary caregiver choose one responder after sending parallel requests"""
1376     with database() as db:
1377         require_owner(db)
1378         assignment = one(db, "SELECT * FROM care_assignment WHERE id = ? AND family_id = ?", (assignment_id,
1379                                                                                             family_id))
1380         if assignment["status"] != "CANDIDATE_ACCEPTED":
1381             raise HTTPException(409, "수락 응답이 온 후보만 최종 확정할 수 있습니다")
1382         item = one(db, "SELECT * FROM care_item WHERE id = ?", (assignment["item_id"],))
1383         timestamp = now()
1384         db.execute("UPDATE care_assignment SET status = 'ACCEPTED', responded_at = ? WHERE id = ?", (timestamp,
1385                                                                                               assignment_id))
1386         db.execute(
1387             """UPDATE care_assignment SET status = 'CANCELED' WHERE item_id = ? AND id <> ?
1388             AND status IN ('PROPOSED', 'CANDIDATE_ACCEPTED', 'RECONFIRMATION_REQUIRED')""",
1389             (item["id"], assignment_id),
1390         )
1391         db.execute("UPDATE care_item SET status = 'ASSIGNED' WHERE id = ?", (item["id"],))
1392         notify(db, assignment["assignee_id"], "물론 담당이 최종 확정됐어요", item["title"],
1393              "IMPORTANT", "ASSIGNMENT_REQUEST", assignment_id)
1394         notify(db, assignment.get("requested_by_member_id") or owner_id(db), "배정이 확정됐어요",
1395              f"(item['title']): 담당자를 최종 확정했어요.", action_type="ASSIGNMENT_RESULT", action_id=assignment_id)
1396         _propagate_routine_assignment(db, item, assignment["assignee_id"], assignment.get("requested_by_member_id"))

```

수락 후보를 최종 담당자로 확정하고 다른 후보 요청을 마감한 뒤 결과 알림을 생성한다.

5 서비스 연동 흐름

Firebase는 별도 데이터 플랫폼이 아니라 서비스 이벤트를 Android 기기에 전달하는 푸시 채널로 사용한다. 일정과 배정 데이터는 백엔드 DB가 관리한다.

기능 이벤트 배정 요청 및 확정 →	DB 알림 notification 저장 →	기기 조회 push token 조회 →	FCM 전송 Firebase Admin →	앱 수신 Capacitor Push →	화면 이동 actionType 분기
단계	구현 내용				연결 지점
알림 생성	기능 이벤트 발생 시 알림 내용과 actionType actionId 저장				main.py notify
토큰 등록	Android에서 받은 FCM 토큰을 인증된 가족 구성원과 연결				POST api push tokens
외부 전달	Firebase Admin SDK가 등록 토큰으로 notification과 data payload 전송				push.py send push
앱 수신	수신 알림을 로컬 알림으로 표시하고 클릭 메타데이터 전달				nativeNotifications.ts
화면 연결	알림 종류에 따라 배정 요청 결과 추천 또는 알림함 화면으로 이동				App.tsx action router

```

ACTUAL SOURCE CODE

DB 알림과 기기 토큰 전달 backend/app/main.py lines 55-60
55 from .push import send_push
56 token_rows = db.execute(
57     "SELECT token FROM push_device_token WHERE family_id = ?" + (" AND member_id = ?" if member_id else ""),
58     (target_family, member_id if member_id else (target_family,)),
59 ).fetchall()
60 send_push([row[0] for row in token_rows], title=100, body=200, action_type, action_id)

Firebase Admin을 통한 FCM 전송 backend/app/push.py lines 29-44
29 def send_push(tokens: list[str], title: str, body: str, action_type: str | None, action_id: str | None) -> None
30 app = _app()
31 if not app or not tokens:
32     return
33 from firebase_admin import messaging
34 data = {"action_type": action_type or "", "action_id": action_id or ""}
35 for token in tokens:
36     try:
37         messaging.send(messaging.Message(
38             token=token,
39             notification=messaging.Notification(title=title, body=body),
40             data=data,
41             app=app)
42         )
43     except Exception:
44         # A stale device token must not break the family action that created the notice.
45         continue

Android FCM 등록과 알림 동작 frontend/src/nativeNotifications.ts lines 52-70
52 // FCM is opt-in until google-services.json and a sending server are configured.
53 if (import.meta.env.VITE_FCM_ENABLED !== "true") {
54     try {
55         console.error('[FCM] registration requested')
56         await PushNotifications.addListener('registration', token => {
57             console.error('[FCM] registration token:', token.value)
58             localStorage.setItem('pushTokenKey', token.value)
59             void syncPushToken()
60         })
61         await PushNotifications.addListener('registrationError', error => {
62             console.error('[FCM] registration error:', error)
63         })
64     }
65     if (onAction) {
66         await PushNotifications.addListener('pushNotificationActionPerformed', event => {
67             const data = event.notification.data ?? {}
68             onAction({
69                 actionType: data.action_type ?? data.actionType,
70                 actionId: data.action_id ?? data.actionId,
71             })
72         })
73     }
74 }

```

DB 알림 생성 이후 Firebase 전송과 Android 수신으로 이어지는 실제 구현 코드이다.

FCM 환경이 설정되지 않은 경우에도 DB 알림과 앱 내부 알림함은 유지되며 외부 푸시 전송만 생략된다.